

Monolithic vs Microservices Architecture?

Monolithic

- Monolithic architecture is a software design approach where all components of an application, such as user management, business logic, and database access, are built and deployed as a single unified application.
- Monolithic architecture is simpler and suitable for small applications.
- **HINDI:-** Monolithic architecture mein poora application ek single codebase aur deployment unit mein hota hai. Agar application ka koi ek module change hota hai to poori application redeploy karni padti hai.

Microservices

- Microservices architecture is a software design approach where an application is divided into multiple small, independent services. Each service is responsible for a specific business function and can be developed, deployed, and scaled independently.
- microservices architecture is more scalable and suitable for large, complex, and enterprise-level applications.
- **HINDI:-** Microservices architecture mein application ko multiple independent services mein divide kiya jata hai. Har service apna codebase, database aur deployment maintain karti hai. Isse independent scaling aur deployment possible hota hai.

Advantages

1. Easy Development:- Ek hi codebase.
2. Easy Deployment:- Sirf ek application deploy karni hoti hai.
3. Debugging Easy:- Sab logs ek jagah milte hain.
4. Faster Initial Development:- Startup projects ke liye best.

Disadvantages

1. Large Codebase:- Application badi hote hi manage karna mushkil.
2. Single Point of Failure:- Agar app crash hua to poora system down.
3. Scaling Problem:- Sirf Orders module busy hai to bhi poori application scale karni padegi.
4. Slow Deployment:- Chhoti change ke liye bhi poora app deploy karna padega.

How do microservices communicate?

- Microservices communicate with each other using **APIs** or **messaging systems**. The most common communication methods are synchronous communication using REST APIs or gRPC, and asynchronous communication using message brokers such as RabbitMQ, Apache Kafka, or Amazon SQS.

1. Synchronous Communication (REST API)
2. Asynchronous Communication (Message Queue)